



Guia Prático: Automação com CI/CD

Por Profa Lia Mariana QA

1. Requisitos Pró-Ativos

Antes de começar, certifique-se de ter o Python instalado. Abra o terminal e instale as ferramentas necessárias (os "equipamentos" do nosso agente):

```
pip install selenium webdriver-manager
```

2. O Código de Teste (`test_login.py`)

Crie um arquivo chamado `test_login.py`.

⚠️ AVISO CRÍTICO DE HARDWARE: > Nas versões atuais do Selenium, você **DEVE** passar o `service` e as `options` explicitamente. Se você esquecer a linha abaixo, o Selenium não conseguirá compreender como iniciar o navegador no ambiente moderno:

```
self.driver = webdriver.Chrome(service=service, options=chrome_options)
```

```
import unittest
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.chrome.options import Options
from webdriver_manager.chrome import ChromeDriverManager

class TestMissaoLogin(unittest.TestCase):
    def setUp(self):
        chrome_options = Options()
        chrome_options.add_argument("--headless")
        chrome_options.add_argument("--no-sandbox")
        chrome_options.add_argument("--disable-dev-shm-usage")

        service = Service(ChromeDriverManager().install())
        self.driver = webdriver.Chrome(service=service, options=chrome_options)
        self.driver.implicitly_wait(10)
```



```
def test_executar_login(self):
    driver = self.driver
    driver.get("https://www.facebook.com/login")
    COLOQUEM AQUI O CODIGO DE LOGIN
    print("Infiltração concluída com sucesso!")

    self.driver.quit()

if __name__ == "__main__":
    unittest.main()
```

3. Configurando o CI/CD (GitHub Actions)

Agora vamos criar o "cérebro" que rodará o código no GitHub.

1. No seu repositório do GitHub, crie uma pasta chamada **.github** (tem que ter o ponto!).
2. Dentro dela, crie outra pasta chamada **workflows**.
3. Dentro de **workflows**, crie o arquivo **main.yml** e cole o conteúdo abaixo:

```
name: Pipeline_Operacao_CI

on: [push] # Gatilho: roda toda vez que você enviar código

jobs:
  executar-testes:
    runs-on: ubuntu-latest # A máquina virtual do QG

    steps:
      - name: Baixando o Código
        uses: actions/checkout@v4

      - name: Instalando o Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.10'

      - name: Instalando Dependências
        run: |
          pip install selenium webdriver-manager

      - name: Rodando a Missão (Testes)
        run: python test_login.py
```



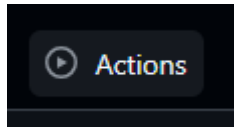
4. Como Validar a Missão

Para ver a mágica acontecer:

Salve tudo e faça o envio para o GitHub:

```
git add .  
git commit -m "Adicionando automação de teste"  
git push origin main
```

1. Vá até o seu repositório no navegador e clique na aba **"Actions"**.



a.

2. Lá você verá um círculo amarelo (em progresso). Se tudo estiver certo, ele ficará **verde**.

Desafio: Mude algo no código para o teste falhar e veja o GitHub te "dar uma bronca" com um sinal **vermelho**.

Notas Importantes:

- **Headless:** É o modo "invisível". Sem ele, o GitHub Actions falha porque não tem uma tela para mostrar o navegador.
- **Requirements:** Em projetos grandes, em vez de `pip install selenium`, usamos um arquivo `requirements.txt`.
- **Sinalizador:** O CI/CD é o seu melhor amigo. Se o código passou ali, você pode dormir tranquilo!